Local LLM Benchmarking

Saved notebook results: local vLLM, Ollama and Spark

Workshop

20 October 2026

Compared Models: Parameters and Capabilities

Model / run	Parameters (billions)	Published capabilities
Gemma 4 E4B Local vLLM; MLX 4-bit	**8B** with embeddings **4.5B effective** (dense)	Text, image, audio input. Coding, reasoning, tools. **128K** token context.
Gemma 4 26B A4B Ollama: `gemma4:26b`	**25.2B total** (MoE) **3.8B active/token**	Text, image input. Coding, reasoning, tools. **256K** token context.
DeepSeek V4 Flash Spark endpoint	**284B total** (MoE) **13B active/token**	Text input. Coding, math, agents; optional thinking. **1M** token context.

E4B effective counts exclude lookup embeddings; Gemma encoders are additional. MoE = Mixture of Experts (subset active per token). Four-bit precision reduces storage, not parameters.

All output text. These runs test text only; endpoint capabilities / context limits are unverified.

Sources: Google Gemma 4; Ollama; DeepSeek release card (284B; transparency card: 285B).

Agenda

1. Setup & Scope
2. Latency
3. Throughput
4. Response Completeness & Text Metrics
5. Audio Transcription
6. LiveCodeBench-Pro
7. Summary

Environment Overview

Local vLLM	`mlx-community/gemma-4-e4b-it-4bit` OpenAI-compatible local endpoint: `localhost:8000`
Ollama	`gemma4:26b` at `localhost:11434`
Spark endpoint	`deepseek-ai/DeepSeek-V4-Flash` Remote endpoint: `spark-0240:8000`

Common prompts

Capital of France; short AI poem; quantum computing;
three laws of robotics; plot of *Inception*.

Model IDs and endpoints are recorded by the notebooks; server software, hardware and quantization are not independently verified here.

What These Runs Compare

Setting	Local vLLM	Ollama	Spark
Requests	5	5	5
Temperature	0.7	0.7	0.7
Maximum tokens	150	800	800
Chat endpoint	`/v1/chat/completions`		

Different models and output budgets

These are sequential smoke benchmarks, not a controlled backend comparison or a concurrent load test. Output lengths and reasoning also differ.

Ollama and Spark include an unmeasured smoke request before the benchmark. The local vLLM notebook records no explicit warm-up. Saved outputs are snapshots; some cells lack execution counts.

Per-Request End-to-End Latency

Prompt	Local vLLM	Ollama	Spark
Capital of France	1.745 s	1.478 s	0.607 s
AI poem	5.491 s	19.619 s	3.686 s
Quantum computing	7.818 s	22.523 s	23.756 s
Robotics laws	7.463 s	14.958 s	8.349 s
Inception	7.794 s	24.565 s	15.920 s

- Spark is fastest for the capital query and poem in these runs.
- Local vLLM returns sooner for the other three prompts, with a much smaller token budget and visibly cut-off answers.

Latency Summary

Metric	Local vLLM	Ollama	Spark
Minimum	1.745 s	1.478 s	0.607 s
Maximum	7.818 s	24.565 s	23.756 s
Mean	6.062 s	16.628 s	10.464 s
Median	7.463 s	19.619 s	8.349 s
Sample std. dev.	2.599 s	9.206 s	9.408 s
Total time	30.312 s	83.142 s	52.318 s

Standard deviation is the sample statistic ($n - 1$). Local vLLM's saved summary uses unrounded timings; the other summaries are computed from displayed request rows. Five different prompts do not establish run-to-run stability.

Token Counts and Throughput

Metric	Local vLLM	Ollama	Spark
Prompt tokens	85	120	60
Completion tokens	565	2,858	1,514
All tokens	650	2,978	1,574
Completion rate (tok/s)	18.64	34.37	28.94
All-token rate (tok/s)	21.44	35.82	30.09

Definitions

Completion rate = sum of completion tokens / sum of request times.
All-token rate includes prompt tokens; it is not generation speed.

End-to-end rates include request overhead and any server-reported reasoning tokens. Different tokenizers and models limit comparability. Ollama has the highest measured completion rate; these runs do not identify the cause.

Response Completeness Matters

Run	Saved benchmark outcomes
Local vLLM	5 HTTP 200 responses. Quantum, robotics and Inception each reach 150 completion tokens and visibly end mid-response. Finish reasons were not retained.
Ollama	5 visible answers. Quantum and Inception report `length` at 800 tokens and are incomplete. Other responses report `stop`.
Spark	5 visible answers; all report `stop`. Completion counts range from 8 to 700.

Ollama retains reasoning text for all five responses; Spark's recorded reasoning fields are empty. The saved results do not quantify reasoning-token counts separately.

Descriptive Text Metrics — Same Formula

	Type-token ratio			Repeated trigram ratio		
Prompt	vLLM	Ollama	Spark	vLLM	Ollama	Spark
Capital of France	1.000	1.000	1.000	0.000	0.000	0.000
AI poem	0.797	0.738	0.704	0.000	0.000	0.000
Quantum computing	0.716	0.732	0.507	0.009	0.011	0.023
Robotics laws	0.661	0.652	0.690	0.018	0.079	0.036
Inception	0.741	0.812	0.599	0.000	0.000	0.003
Mean	0.783	0.787	0.700	0.005	0.018	0.012

Computed from visible benchmark answers only. Words = lowercase whitespace-separated tokens. TTR = unique words / words. Repetition = trigram occurrences beyond the first / all trigram positions. Means are unweighted across five prompts.

These describe text, not correctness. Length and truncation affect TTR. Historical Spark quality queries in the vLLM notebook are excluded.

Visible Answer Length

Words	Local vLLM	Ollama	Spark
Capital of France	6	6	6
AI poem	74	61	81
Quantum computing	109	97	521
Robotics laws	115	141	171
Inception	108	64	344
Mean	82.4	73.8	224.6

- Spark's longer answers help explain its longer request times on some prompts.
- Ollama's completion counts include reasoning; visible word counts do not.
- Local vLLM and Ollama lengths include incomplete answers.

Qualitative Review of Saved Answers

Prompt	Local vLLM	Ollama	Spark
Capital	Concise Paris answer	Concise Paris answer	Concise Paris answer
AI poem	12 lines, 3 stanzas	8 lines, 2 stanzas	12 lines, 3 stanzas
Quantum	Maze analogy; cut off	Classical bits explanation; cut off	Long librarian/genie analogy with headings and table
Robotics	Altered law wording; third law cut off	Three laws plus Zeroth Law context	Three laws plus fictional-origin and Zeroth Law context
Inception	Setup; cut off mid-sentence	Premise; cut off mid-sentence	Full-length synopsis with nested dream levels

Completeness and lexical diversity do not prove factual accuracy. The local robotics response changes the law wording; the quantum analogies warrant a separate factual review. No systematic correctness evaluation was performed.

Dutch Audio: Models, Locations and WER

UMC Utrecht: Jeroen Hendrikse, radiologieverslag
473.6 seconds of Dutch speech; 1,350 normalized reference words.

Model	Runtime and location	WER ↓
Whisper large-v3	vLLM Metal, local Mac `localhost:8000`	12.00%
Gemma 4 E4B IT	MLX-VLM, local Mac Direct Python inference	**7.78%**
Qwen3-Omni 30B A3B Instruct	vLLM, NVIDIA Spark `spark-0240:8000`	8.44%

Gemma has the lowest WER on this recording: 35% fewer word errors than the recorded Whisper run. Qwen trails Gemma by 9 word errors.

Sources: saved Gemma/Qwen notebooks and earlier recorded Whisper run (outputs now cleared). Both curated TXT and SRT references give the same scores.

Audio Evaluation: Scoring and Comparison Scope

$$\mathrm{WER} = \frac{\text{substitutions} + \text{deletions} + \text{insertions}}{\text{reference words}}$$

Model	Substitutions	Deletions	Insertions	Total
Whisper large-v3	86	41	35	162
Gemma 4 E4B IT	69	21	15	105
Qwen3-Omni	73	15	26	114

- Same audio and references. Scoring ignores case and punctuation, splits hyphens, and retains fillers and repetitions.
- Gemma/Qwen: identical prompt and 18 chunks (max. 28 s). Whisper: complete MP3 to its transcription API. All use temperature 0.
- One recording, different inference pipelines. Results compare the tested setups, without isolating model or hardware effects.

LiveCodeBench-Pro: Methodology

Judge architecture: linux/arm64.
Workshop subset: 10 identical problems per model.
One C++ solution attempt; temperature 0; token budget 4096.
Dataset revision: `adebffce047d`.
Upstream commit: `d614eef4f95b`.

Acceptance = accepted / selected problems. Missing or failed evaluations do not count as accepted.
Subset scores are not full-benchmark leaderboard scores. Inference latency excludes judging.
Spark endpoint: `http://spark-0240:9000`.
Source: LiveCodeBench-Pro

LiveCodeBench-Pro: Three-Model Results

Metric	Local vLLM	Ollama	Spark
Accepted / selected	Not run	0/10	4/10
Acceptance	Not run	0.0%	40.0%
Completed judging	Not run	0/10	10/10
Failures	Not run	10	0
Mean inference latency	Not run	103.73 s	25.32 s
Truncated responses	Not run	10	0

Completed judging includes compilation errors and definitive code verdicts. Failures include inference errors, missing code, judge errors and timeouts.
Incomplete runs remain visible in the selected-problem denominator. Truncations may also have a judge verdict.
Ollama: gemma4:26b (2026-10-08)
Spark: deepseek-ai/DeepSeek-V4-Flash (2026-10-20)

LiveCodeBench-Pro: Verdict Breakdown

Verdict	Local vLLM	Ollama	Spark
Accepted	Not run	0	4
Compilation Error	Not run	0	4
No Code	Not run	10	0
Wrong Answer	Not run	0	2
Not evaluated	Not run	0	0

Different models, quantization and reasoning behavior limit backend comparisons.

What the Saved Results Support

- **Local vLLM:** lowest mean latency (6.062 s), but a 150-token budget and three visibly incomplete answers.
- **Ollama:** highest completion rate (34.37 tok/s), longest total time (83.142 s), and two explicitly truncated answers.
- **Spark:** fastest capital response (0.607 s); all responses finish with `stop`. Mean latency is 10.464 s and sample standard deviation is 9.408 s.

Interpretation

These snapshots show tradeoffs in output length, reasoning and completion. They do not establish the best backend, equal answer quality, or predictable latency under load.

A Fairer Follow-Up Benchmark

1. Match prompts, token budgets and reasoning settings; compare the same model where possible.
2. Record server versions, hardware, quantization and warm-up conditions.
3. Repeat each prompt; retain full answers, finish reasons and usage details.
4. Separate completion throughput, time to first token and concurrent serving capacity.
5. Evaluate correctness and completeness alongside descriptive text metrics.

Sources and Questions

- `local-llm-vllm.ipynb`: local benchmark output in cell index 1.
- `local-llm-ollama.ipynb`: benchmark in cell index 8; visible answers in cell index 10.
- `local-llm-spark-cluster.ipynb`: benchmark in cell index 8; visible answers in cell index 10.

Scope of this update

Smoke-test metrics use saved notebook outputs from 7 October 2026. Coding results use Ollama's 8 October run and Spark's 20 October run on port 9000. Historical Spark comparisons and separate completion-endpoint queries inside the local vLLM notebook are excluded.

Questions?